Source code

```
restaurant_absa_project/
│
├── train.py
├── evaluate_model.py
├── predict.py
├── model.py
├── preprocess.py
├── utils.py
├── data_loader.py
│
├── requirements.txt
├── README.md
├── example_configs.json
│
└── data/
    ├── train.jsonl
    ├── dev.jsonl
    └── test.jsonl
```

RESTAURANT ABSA PROJECT

1.) train.py

```
import json
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from transformers import AutoTokenizer
from tqdm import tqdm

from data_loader import ABSADataset
from model import ABSAClassifier
from utils import set_seed, get_device
from preprocess import simple_clean
import os

def collate_batch(batch, tokenizer, max_length=128):
texts = [simple_clean(x["text"]) for x in batch]
enc = tokenizer(texts, padding=True, truncation=True, max_length=max_length,
return_tensors="pt")
return enc, batch

def majority_polarity(aspects):
counts = {"positive": 0, "neutral": 0, "negative": 0}
for a in aspects:
pol = a.get("polarity", "neutral").lower()
if pol in counts:
counts[pol] += 1
return max(counts, key=counts.get)
```

```
def polarity_index(p):
if p == "negative":
return 0
if p == "neutral":
return 1
return 2 # positive

def main():
set_seed(42)
cfg = json.load(open("example_configs.json"))
device = get_device(cfg["device"])

tokenizer = AutoTokenizer.from_pretrained(cfg["model_name"])

train_ds = ABSADataset("data/train.jsonl")
dev_ds = ABSADataset("data/dev.jsonl")

train_loader = DataLoader(
train_ds,
batch_size=cfg["batch_size"],
shuffle=True,
collate_fn=lambda b: collate_batch(b, tokenizer, cfg["max_length"])
)

dev_loader = DataLoader(
dev_ds,
batch_size=cfg["batch_size"],
shuffle=False,
collate_fn=lambda b: collate_batch(b, tokenizer, cfg["max_length"])
)

model = ABSAClassifier().to(device)
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=cfg["lr"])

# create checkpoint folder
os.makedirs("checkpoints", exist_ok=True)

for epoch in range(cfg["epochs"]):
print(f"\nStarting Epoch {epoch}...")
model.train()

for enc, raw in tqdm(train_loader, desc=f"Epoch {epoch}"):
input_ids = enc["input_ids"].to(device)
mask = enc["attention_mask"].to(device)

gold = [polarity_index(majority_polarity(x["aspects"])) for x in raw]
gold = torch.tensor(gold).to(device)

logits = model(input_ids, mask)
loss = F.cross_entropy(logits, gold)

optimizer.zero_grad()
loss.backward()
optimizer.step()

# ------- Evaluation -------
model.eval()
preds, gts = [], []

with torch.no_grad():
for enc, raw in dev_loader:
input_ids = enc["input_ids"].to(device)
mask = enc["attention_mask"].to(device)
```

```
gold = [polarity_index(majority_polarity(x["aspects"])) for x in raw]
pred = model(input_ids, mask).argmax(dim=-1).cpu().tolist()

preds.extend(pred)
gts.extend(gold)

correct = sum([1 for p, g in zip(preds, gts) if p == g])
accuracy = correct / len(gts)

print(f"Dev Accuracy: {accuracy:.4f}")

# SAVE MODEL HERE (correct indentation)
torch.save(model.state_dict(), "checkpoints/best.pt")
print("Model saved to checkpoints/best.pt")

if __name__ == "__main__":
main()
```

2). Evaluate_model.py

```
import json
import torch
from torch.utils.data import DataLoader
from transformers import AutoTokenizer

from data_loader import ABSADataset
from preprocess import simple_clean
from model import ABSAClassifier

def collate_batch(batch, tokenizer, max_length=128):
texts = [simple_clean(x["text"]) for x in batch]
enc = tokenizer(texts, padding=True, truncation=True, max_length=max_length,
return_tensors="pt")
return enc, batch

def polarity_index(p):
if p == "negative":
return 0
if p == "neutral":
return 1
return 2

def main():
cfg = json.load(open("example_configs.json"))

tokenizer = AutoTokenizer.from_pretrained(cfg["model_name"])

model = ABSAClassifier()
model.load_state_dict(torch.load("checkpoints/best.pt", map_location="cpu"))
```

```
model.eval()

ds = ABSADataset("data/test.jsonl")
loader = DataLoader(
ds,
batch_size=16,
shuffle=False,
collate_fn=lambda b: collate_batch(b, tokenizer)
)

preds, gts = [], []

for enc, raw in loader:
logits = model(enc["input_ids"], enc["attention_mask"])
pred = logits.argmax(dim=-1).tolist()
preds.extend(pred)

gold = [polarity_index(a["polarity"]) for x in raw for a in x["aspects"]]
gts.extend(gold)

correct = sum([1 for p, g in zip(preds, gts) if p == g])
accuracy = correct / len(gts)

print("Test Accuracy:", accuracy)

if __name__ == "__main__":
main()
```

3). predict.py

```
import torch
from transformers import AutoTokenizer
from model import ABSAClassifier
from preprocess import simple_clean

# Load tokenizer and model
tokenizer = AutoTokenizer.from_pretrained("bert-base-multilingual-cased")
model = ABSAClassifier()

# Load trained weights
model.load_state_dict(torch.load("checkpoints/best.pt", map_location="cpu"))
model.eval()

def polarity_label(idx):
if idx == 0:
return "negative"
if idx == 1:
return "neutral"
return "positive"

def predict_sentiment(text):
text = simple_clean(text)
enc = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128)
with torch.no_grad():
logits = model(enc["input_ids"], enc["attention_mask"])
pred = logits.argmax(dim=-1).item()
return polarity_label(pred)

# Test your model here:
if __name__ == "__main__":
```

```
user_text = input("Enter a restaurant review: ")
prediction = predict_sentiment(user_text)
print("\nPredicted Sentiment:", prediction)
```

4).model.py

```
import torch
import torch.nn as nn
from transformers import AutoModel

class ABSAClassifier(nn.Module):
def __init__(self, pretrained_name="bert-base-multilingual-cased", hidden_dim=768,
num_polarities=3):
super().__init__()
self.encoder = AutoModel.from_pretrained(pretrained_name)
self.classifier = nn.Linear(hidden_dim, num_polarities)

def forward(self, input_ids, attention_mask):
output = self.encoder(input_ids=input_ids, attention_mask=attention_mask, return_dict=True)
pooled = output.last_hidden_state[:, 0]
logits = self.classifier(pooled)
return logits
```

6).preprocess.py

```
import re

def simple_clean(text):
text = text.strip()
text = text.replace("\n", " ")
text = re.sub(r"\s+", " ", text)
return text
```

7). utils.py

```
import random
import torch
import numpy as np

def set_seed(seed=42):
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
if torch.cuda.is_available():
torch.cuda.manual_seed_all(seed)

def get_device(prefer='cpu'):
if prefer == 'cuda' and torch.cuda.is_available():
return torch.device('cuda')
return torch.device('cpu')
```

8).data_loader

```
import json
from torch.utils.data import Dataset
from transformers import AutoTokenizer

class ABSADataset(Dataset):
def __init__(self, path, tokenizer_name='bert-base-multilingual-cased', max_length=128):
self.samples = []
with open(path, 'r', encoding='utf8') as f:
for line in f:
if line.strip():
self.samples.append(json.loads(line.strip()))

self.tokenizer = AutoTokenizer.from_pretrained(tokenizer_name)
self.max_length = max_length

def __len__(self):
return len(self.samples)

def __getitem__(self, idx):
obj = self.samples[idx]
return {
"id": obj.get("id", idx),
"text": obj.get("text", ""),
"aspects": obj.get("aspects", [])
}
```

9). requirements.txt

```
torch>=1.13
transformers>=4.30
datasets
tqdm
scikit-learn
sentencepiece
numpy
pandas
python-dotenv
```

10).README.md

Restaurant ABSA (Multilingual) - Project Code

This repository contains the starter code for the Restaurant Aspect-Based Sentiment Analysis (ABSA) project with a multilingual augmentation (translation + consistency loss).

Structure

- `requirements.txt` — Python packages required
- `data_loader.py` — Loads and prepares SemEval-style datasets
- `preprocess.py` — Text preprocessing utilities
- `model.py` — PyTorch model wrapper (mBERT / XLM-R)
- `train.py` — Training script including translation-augmentation + consistency loss
- `evaluate.py` — Evaluation and metrics
- `utils.py` — Helper utilities (seed, device)
- `example_configs.json` — example hyperparameters

Notes

- This is starter code meant to run in a Colab or local GPU environment.
- Datasets: Use SemEval ABSA restaurant dataset (or a similar restaurant review dataset). The loader expects JSONL or TSV with columns: `id`, `text`, `aspects` where `aspects` is a list of {"term":..., "category":..., "polarity":...}.
- For translation augmentation you can either:

1. Provide pre-translated pairs (English↔Target) in the dataset, or
2. Hook an external translation API and set `use_online_translation` in `example_configs.json` (not enabled by default).

How to run (example)

1. Create a virtualenv and install requirements:

```bash
pip install -r requirements.txt
```

11).example_configs.json

```
{
"model_name": "bert-base-multilingual-cased",
"max_length": 128,
"batch_size": 16,
"lr": 2e-5,
"epochs": 3,
"consistency_weight": 1.0,
"device": "cpu",
"use_online_translation": false
}
```

---DATA/

1.train.jsonl


```
{"id": 4, "text": "Food quality was great but I didn't like the ambience.", "aspects":
[{"term":"food","category":"food","polarity":"positive"},{"term":"ambience","category":"ambience
","polarity":"negative"}]}
{"id": 5, "text": "Service was average.", "aspects":
[{"term":"service","category":"service","polarity":"neutral"}]}
```


2.dev.jsonl


```
{"id": 6, "text": "The staff was rude but the food tasted amazing.", "aspects":
[{"term":"staff","category":"service","polarity":"negative"},{"term":"food","category":"food","polar
ity":"positive"}]}
{"id": 7, "text": "Ambience was relaxing.", "aspects":
[{"term":"ambience","category":"ambience","polarity":"positive"}]}
```


3.test.jsonl


```
{"id": 1, "text": "The food was delicious but service was slow.", "aspects":
[{"term":"food","category":"food","polarity":"positive"},{"term":"service","category":"service","pol
arity":"negative"}]}
{"id": 2, "text": "Nice ambience and friendly staff.", "aspects":
[{"term":"ambience","category":"ambience","polarity":"positive"},{"term":"staff","category":"servi
ce","polarity":"positive"}]}
{"id": 3, "text": "The price was too high for the small portion.", "aspects":
[{"term":"price","category":"price","polarity":"negative"},{"term":"portion","category":"food","pol
arity":"negative"}]}
```